

# Rancangan SSO — SSB sebagai Identity Provider (IdP)

Status: **Tahap 1 & 2 SELESAI** (endpoint OAuth aktif, reuse login existing) — Tahap 3 berikutnya Tanggal: 2026-06-06 Aplikasi utama (IdP): **SSB-PROJECT** (Laravel 8 / PHP 8) Client awal (SP): **ESS, Warehouse**

## 1. Tujuan

Menjadikan **SSB-PROJECT** sebagai pusat **Single Sign-On (SSO)** sehingga aplikasi lain (ESS, Warehouse, dan app berikutnya) bisa memakai satu kali login. Implementasi harus **bersifat additive** — tidak mengganggu proses login, API, dan integrasi yang sudah berjalan.

### Keputusan arsitektur (sudah disepakati)

| Topik          | Keputusan                                                                             |
|----------------|---------------------------------------------------------------------------------------|
| Protokol       | <b>OAuth2 / OIDC via Laravel Passport</b>                                             |
| Manajemen role | <b>SSB hanya autentikasi (siapa user). Role/otorisasi dikelola lokal di tiap app.</b> |

## 2. Kondisi aplikasi saat ini (baseline)

- **Framework:** Laravel 8, PHP 8.0
- **Auth web:** session-based ( `laravel/ui` , `LoginController` ) — login pakai **NIK + password**
- **Auth API:** **Laravel Sanctum** ( `AuthController@login` → Bearer token)
- **Otorisasi:** Spatie Permission (roles & permissions)
- **Integrasi eksternal yang sudah ada:**
  - `ServiceTokenController` — token permanen untuk aplikasi eksternal
  - `MediaController` — serve file ke aplikasi eksternal (butuh service token)
- **Master karyawan via API eksternal** (nama di-snapshot, bukan tabel lokal)

Semua hal di atas **TETAP berjalan** dan tidak diubah oleh rancangan ini.

## 3. Prinsip: tanpa mengganggu proses existing

| Yang TETAP jalan (tidak disentuh)                        | Yang DITAMBAH (baru, terpisah)                                             |
|----------------------------------------------------------|----------------------------------------------------------------------------|
| Login web session ( <code>LoginController</code> )       | Endpoint OAuth2: <code>/oauth/authorize</code> , <code>/oauth/token</code> |
| <code>AuthController@login</code> ( Sanctum token)       | Endpoint <code>/oauth/userinfo</code>                                      |
| <code>ServiceToken</code> & <code>MediaController</code> | Halaman login SSO (reuse session existing)                                 |
| Guard <code>web</code> & <code>api</code> ( Sanctum)     | Guard baru <code>oauth</code> ( Passport), berdampingan                    |
| Tabel <code>personal_access_tokens</code> ( Sanctum)     | Tabel <code>oauth_*</code> ( Passport) + <code>sso_clients</code>          |

Strategi: **route baru, tabel baru, controller baru, guard baru**. Flow lama nol perubahan, sehingga setiap tahap bisa di-rollback tanpa risiko.

## 4. Pembagian tanggung jawab

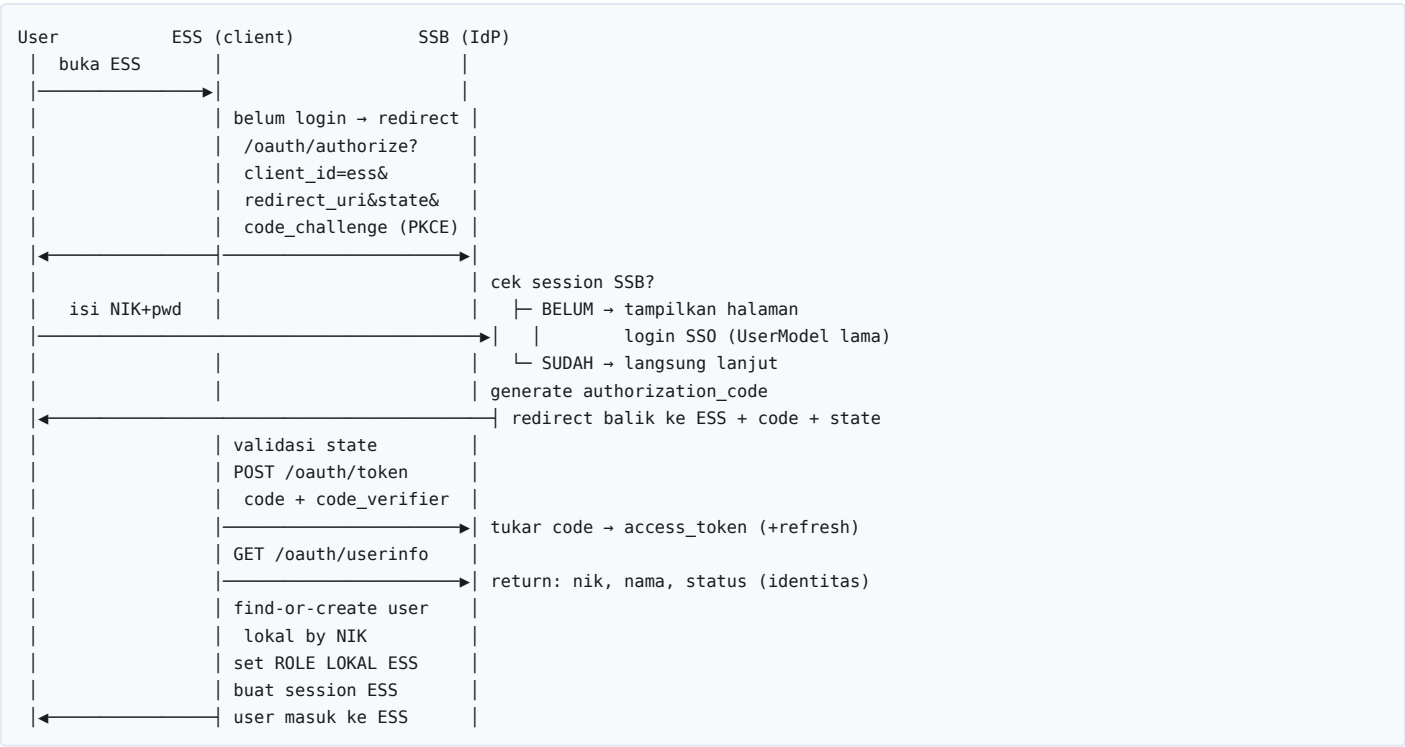
|                            | SSB (IdP)                       | ESS / Warehouse (Client)  |
|----------------------------|---------------------------------|---------------------------|
| Autentikasi (siapa user)   | pusat — validasi NIK + password | menerima hasil            |
| Identitas user (NIK, nama) | sumber kebenaran                | sinkron by NIK            |
| Role / permission          | tidak mengatur                  | dikelola lokal tiap app   |
| Session aplikasi           | session SSB (untuk SSO)         | session lokal app sendiri |

Karena role bersifat lokal, kontrak SSO menjadi sederhana: SSB tidak perlu tahu hak akses tiap app.

## 5. Arsitektur



## 6. Workflow Login (Authorization Code + PKCE)



**Kunci true SSO:** `/oauth/authorize` membaca **session SSB**. Sekali user login di salah satu app, app berikutnya tidak perlu ketik password lagi — langsung mendapat `authorization_code`.

## 7. Kontrak `/oauth/userinfo` (identitas saja)

```
{
  "sub": "12345",           // NIK = identitas universal antar app
  "nik": "12345",
  "name": "Budi Santoso",  // snapshot dari API karyawan
  "email": null,
  "is_active": true        // status kepegawaian (dari API karyawan)
}
```

Tidak ada field `roles`. Tiap app memetakan NIK → role lokalnya sendiri.

## 8. Workflow Logout

Tahap awal cukup **front-channel logout** (per app):

```
User klik logout di ESS
→ ESS hapus session lokal
→ redirect ke SSB /sso/logout?redirect=<ess>
→ SSB hapus session SSB + revoke token app tsb
→ redirect balik ke ESS
```

**Single Logout (SLO) penuh** — back-channel ke semua app — dapat menyusul di tahap lanjutan.

## 9. Penambahan database (semua tabel BARU)

- Tabel bawaan Passport (via `php artisan passport:install`): `oauth_clients`, `oauth_access_tokens`, `oauth_refresh_tokens`, `oauth_auth_codes`, `oauth_personal_access_clients`.
- Tabel milik sendiri:
  - `sso_clients` — metadata app client: nama, logo, **whitelist redirect\_uri**, status aktif. Dipakai untuk admin panel kelola aplikasi yang boleh memakai SSO.

`personal_access_tokens` (Sanctum) **tetap dipakai** untuk API existing — tidak bentrok.

## 10. Yang dikelola di tiap Client (ESS / Warehouse)

1. Tabel `users` lokal minimal: `nik` (unik, kunci sinkron), `name`, `last_login`.
2. Tabel role/permission **milik app sendiri** (boleh Spatie lagi atau tabel sederhana), di-mapping ke `nik`.
3. Saat callback SSO: `find-or-create user by nik` → set role lokal app → buat session.
4. **First login** (NIK belum punya role di app): arahkan ke halaman "akses belum diberikan / hubungi admin app", atau auto-assign role default. (Keputusan masing-masing admin app.)

## 11. Checklist implementasi

Di SSB (IdP) — semua additive

- ☒ `composer require laravel/passport "^10.4"` (v10.4.2 — wajib seri 10.x untuk Laravel 8) **[Tahap 1]**
- ☒ `php artisan migrate` → tabel `oauth_*` terbuat **[Tahap 1]**
- ☒ `php artisan passport:keys` → kunci di `storage/oauth-*.key` (sudah ter-ignore via `/storage/*.key`) **[Tahap 1]**
- ☒ Tambah guard `oauth` di `config/auth.php` (driver `passport`; `web` / `api` TIDAK diubah) **[Tahap 1]**
- ☒ Route `/oauth/authorize` & `/oauth/token` (registrasi `Passport::routes()`); reuse session + `LoginController` existing untuk halaman login **[Tahap 2]**
- ☒ Publish view consent Passport + atur masa berlaku token (access 15 mnt, refresh 30 hari) **[Tahap 2]**
- ☐ Route `/oauth/userinfo` → kembalikan identitas saja (NIK, nama, status) dari API karyawan **[Tahap 3]**
- ☐ Tabel `sso_clients` + admin panel: daftar `client_id`, `secret`, **whitelist** `redirect_uri` + flag trusted (skip consent first-party) **[Tahap 3]**
- ☐ Route `/sso/logout` (front-channel logout) **[Tahap 5]**

Di tiap Client (ESS, Warehouse)

- ☐ `composer require laravel/socialite` + provider custom "ssb"
- ☐ Config `client_id`, `client_secret`, `redirect_uri`, base URL SSB
- ☐ Callback: find-or-create by NIK + mapping role lokal + session
- ☐ Middleware `redirect-to-SSO` + tombol "Login dengan SSB"

12. Keamanan (wajib)

- **PKCE** untuk semua client (lindungi authorization code).
- **Whitelist** `redirect_uri` per client — cegah open redirect.
- **HTTPS** wajib di semua endpoint.
- Parameter **state** anti-CSRF pada alur authorize.
- Access token **short-lived** + refresh token; **revoke** saat logout.
- **Rate-limit** `/oauth/token` dan halaman login SSO.
- Validasi status kepegawaian (`is_active`) — user nonaktif ditolak di IdP.

13. Roadmap bertahap (aman, bisa rollback)

| Tahap | Output                                                                                                                | Dampak ke app lama  |
|-------|-----------------------------------------------------------------------------------------------------------------------|---------------------|
| 1     | Pasang Passport + tabel <code>oauth_*</code> + guard <code>oauth</code> (belum diaktifkan ke publik) — <b>SELESAI</b> | nol (terverifikasi) |
| 2     | Aktifkan <code>/oauth/authorize</code> & <code>/oauth/token</code> , reuse login session existing — <b>SELESAI</b>    | nol (terverifikasi) |
| 3     | <code>/oauth/userinfo</code> (identitas) + <code>sso_clients</code> + admin panel                                     | nol                 |
| 4     | Integrasi 1 client pilot (ESS) — terisolasi                                                                           | nol                 |
| 5     | Warehouse menyusul + front-channel logout                                                                             | nol                 |
| 6     | (Opsional) Single Logout penuh & migrasi API existing bertahap                                                        | dikontrol           |

Setiap tahap tidak menyentuh: login web lama, Sanctum API, ServiceToken, MediaController.

## 14. Catatan Implementasi Tahap 1 (selesai 2026-06-06)

### Yang dikerjakan:

```
composer require laravel/passport "^10.4" # terpasang v10.4.2
php artisan migrate                       # buat 5 tabel oauth_*
php artisan passport:keys --force         # storage/oauth-private.key & oauth-public.key
```

Plus konfigurasi (additive) di `config/auth.php` :

```
'oauth' => [
    'driver' => 'passport',
    'provider' => 'users',
],
```

### Keputusan teknis penting:

1. **Versi Passport dikunci ke seri 10.x** ( ^10.4 ). Passport 11+ butuh Laravel 9+, sedangkan project ini Laravel 8.83 → wajib v10.x.
2. **Passport::routes()** **SEGAJA belum dipanggil**. Akibatnya endpoint `/oauth/authorize`, `/oauth/token`, dll **belum terdaftar** (terverifikasi via `php artisan route:list`). Ini menjaga Tahap 1 benar-benar "belum aktif ke publik". Registrasi route = Tahap 2.
3. **User model TIDAK diubah**. Model masih memakai `Laravel\Sanctum\HasApiTokens` sehingga API existing (login Sanctum, ServiceToken, MediaController) tetap utuh. Alur Authorization Code menerbitkan token lewat OAuth server Passport, bukan lewat trait model, jadi trait Passport belum diperlukan.

Catatan lanjutan: bila nanti butuh helper token Passport pada model (mis. `$user->tokens()`), import dengan alias agar tidak bentrok dengan Sanctum: `use Laravel\Passport\HasApiTokens as HasPassportTokens;`

4. **Kunci enkripsi aman dari git** — `.gitignore` sudah memuat `/storage/*.key` sebelum perubahan ini, jadi `oauth-private.key` tidak akan ter-commit.

### Verifikasi non-disruptif (sudah dijalankan):

- `route:list | grep oauth` → tidak ada route OAuth Passport (hanya `api/oauth2-callback` milik Swagger yang memang sudah ada).
- Route login lama (`/`, `api/login`, `hrd/auth/login`) tetap ada.
- Guard terdaftar: `web`, `api`, `oauth`, `sanctum`.

**File yang berubah:** `composer.json`, `composer.lock`, `config/auth.php` (+ `vendor/` dan `storage/oauth-*.key` yang ter-ignore git).

## 15. Catatan Implementasi Tahap 2 (selesai 2026-06-06)

**Tujuan:** mengaktifkan endpoint OAuth2 dan membuktikan alur authorize me-reuse halaman login yang sudah ada — tanpa mengubah `LoginController`.

### Yang dikerjakan:

1. Registrasi route Passport + masa berlaku token di `app/Providers/AuthServiceProvider.php` :

```
use Laravel\Passport\Passport;

public function boot()
{
    $this->registerPolicies();
    // ... gate existing ...
}
```

```

Passport::routes(); // aktifkan /oauth/*
Passport::tokensExpireIn(now()->addMinutes(15)); // access token pendek
Passport::refreshTokensExpireIn(now()->addDays(30)); // refresh token
Passport::personalAccessTokensExpireIn(now()->addDays(7));
}

```

2. `php artisan vendor:publish --tag=passport-views` → layar consent di `resources/views/vendor/passport/`.

3. Membuat client uji (tipe **public/PKCE**, tanpa secret — sesuai prinsip keamanan):

```

php artisan passport:client --public --name="ESS (dev)" \
  --redirect_uri="http://localhost:8001/auth/ssb/callback" --no-interaction
# Client ID: 1 (redirect bisa diganti saat integrasi ESS nyata di Tahap 4)

```

### Cara kerja reuse login (kenapa `LoginController` tak perlu diubah):

- Route `/oauth/authorize` otomatis bermiddleware `web, auth` (terverifikasi via `gatherMiddleware()` → `web, auth`).
- Guest yang membuka `/oauth/authorize` dilempar oleh middleware `Authenticate` ke login utama (`APP_URL`), dan URL tujuan disimpan di session (`url.intended`).
- Setelah login, Laravel memakai `redirect()->intended()` yang **memprioritaskan** `url.intended` di atas `redirectTo()` (`/home`) milik `LoginController`, sehingga user kembali ke `/oauth/authorize` dan alur berlanjut.

### Contoh URL authorize (yang nanti dipanggil ESS):

```

GET /oauth/authorize?
  client_id=1&
  redirect_uri=http://localhost:8001/auth/ssb/callback&
  response_type=code&
  scope=&
  state=<random>&
  code_challenge=<base64url(sha256(verifier))>&
  code_challenge_method=S256

```

### Verifikasi (sudah dijalankan):

- `route:list` → `/oauth/authorize`, `/oauth/token`, `/oauth/token/refresh`, `/oauth/tokens` muncul.
- Middleware `/oauth/authorize` = `web, auth`.
- Client uji tersimpan: `id=1`, `name="ESS (dev)"`, `public(secret null)=yes`, `revoked=0`.
- App tetap boot; route & login existing tidak berubah.

### Belum dikerjakan (sengaja, masuk Tahap 3):

- `/oauth/userinfo` (identitas dari API karyawan).
- Tabel `sso_clients` + admin panel + flag **trusted** untuk skip layar consent pada app first-party (saat ini consent screen Passport masih tampil).
- Pembatasan route manajemen client bawaan Passport (`/oauth/clients`, `/oauth/tokens`) — akan diganti admin panel sendiri.

**File yang berubah:** `app/Providers/AuthServiceProvider.php` (+ `resources/views/vendor/passport/**` hasil publish, + 1 row `oauth_clients` di DB).

## 16. Glosarium

- **IdP (Identity Provider):** aplikasi pusat yang mengautentikasi user — di sini SSB.
- **SP / Client (Service Provider):** aplikasi yang mempercayakan login ke IdP — ESS, Warehouse.
- **Authorization Code + PKCE:** alur OAuth2 paling aman untuk web app, code ditukar token via back-channel.

- **Front-channel logout:** logout via redirect browser (per app).
- **Single Logout (SLO):** logout serentak di semua app via back-channel.